

Proyecto Final: Mapa de Sabores

Desarrollo de Sistemas Web

Alumno: Diego Rafael Guaraz

Docente / Cátedra: Proyecto Final Desarrollo de Sistemas Web

Documento Ejecutivo de Presentación de Proyecto

1. Resumen

Mapa de Sabores es un sistema web interactivo de descubrimiento gastronómico basado en una arquitectura híbrida de **Base de Datos Relacional + Inteligencia Artificial (IA)**.

El proyecto resuelve el problema del maridaje e innovación culinaria (*flavor pairing*) mediante la representación en forma de un **Grafo de Sabores** dinámico. Permite a chefs, estudiantes de gastronomía y aficionados explorar combinaciones de ingredientes basados en afinidad organoléptica y química, consultar recetas reales integradas y recibir explicaciones culinarias generadas en lenguaje natural por modelos de lenguaje (LLM).

Principales Pilares de Ingeniería:

1. **Certeza en el Core (Base Relacional Estable):** La red de sabores reside en una base de datos PostgreSQL optimizada con índices compuestos bidireccionales. La estructura del grafo se define por datos curados y validados, garantizando consistencia, respuestas instantáneas (< 10ms) y cero alucinaciones en la navegación.
 2. **Pipeline de Datos Sintéticos Offline:** Un pipeline de ingeniería de prompts sobre LLMs compila y cura un dataset inicial de ~250 ingredientes y ~1,500 pares de afinidad.
 3. **Capa de IA Desacoplada e Intercambiable:** Un servicio backend agnóstico en FastAPI permite alternar entre proveedores cloud (Google Gemini, OpenAI) y ejecución 100% local (Ollama / Llama 3.2 3B).
 4. **Visualización React en 2D:** Renderizado dinámico de nodos y aristas mediante HTML5 Canvas (`react-force-graph-2d`) con experiencia de usuario fluida y paneles laterales descriptivos.
-

2. Planteamiento del Problema y Objetivos

2.1 Problema Identificado

Tradicionalmente, el descubrimiento de combinaciones de ingredientes ha dependido de la intuición empírica o de enciclopedias culinarias estáticas. Aunque

existen teorías científicas de maridaje de sabores (compartir compuestos aromáticos clave), no existen herramientas web abiertas e interactivas en español que combinen: * Exploración visual intuitiva en forma de grafo dinámico. * Explicaciones organolépticas personalizadas en lenguaje natural. * Búsqueda en tiempo real de recetas aplicadas.

2.2 Objetivos del Proyecto

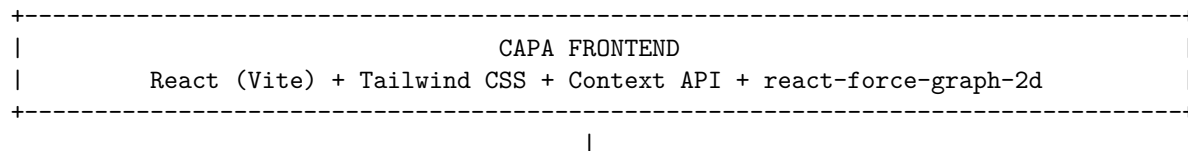
- **Objetivo General:** Desarrollar una aplicación web full-stack funcional y escalable que permita explorar redes de sabores e interacciones de ingredientes asistida por Inteligencia Artificial.
- **Objetivos Específicos:**
 1. Diseñar un esquema relacional optimizado en PostgreSQL para modelar grafos bidireccionales de afinidad.
 2. Implementar un pipeline offline en Python para la generación, validación y sanitización de un dataset sintético de afinidades culinarias.
 3. Crear una API REST en FastAPI con arquitectura limpia y abstracción del proveedor de LLM.
 4. Desarrollar una interfaz de usuario interactiva en React utilizando la librería `react-force-graph-2d`.
 5. Integrar autenticación JWT para áreas personalizadas de usuarios (favoritos y recetas).

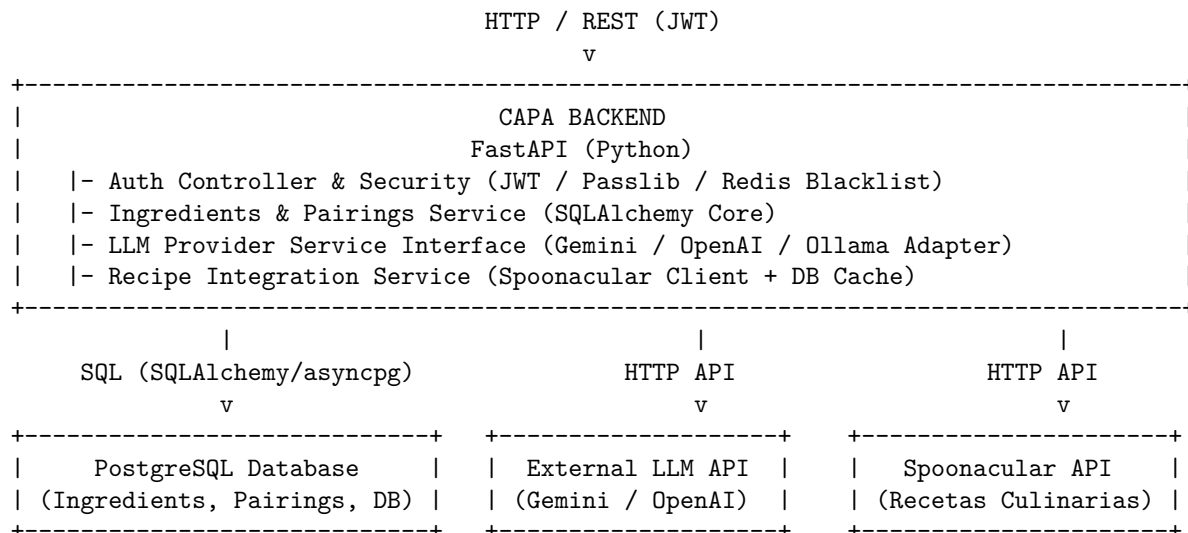
2.3 Justificación de Decisiones de Diseño

- **PostgreSQL vs. Neo4j:** Se optó por PostgreSQL debido a que en una red de 300 a 1,000 ingredientes las consultas de 1 o 2 saltos (*hops*) no justifican la sobrecarga operativa de un motor de grafos nativo como Neo4j. Mediante índices compuestos y ordenamiento de IDs (`ingredient_a_id < ingredient_b_id`), Postgres resuelve estas consultas de forma directa en **< 10ms**, reduciendo drásticamente los costos de infraestructura.
- **Arquitectura Híbrida de IA:** La IA no actúa como la base de datos (evitando latencia y respuestas inestables), sino como un potenciador en dos fases: compilación del dataset offline y generación de prosa gastronómica online bajo demanda.

3. Arquitectura y Modelo de Datos (Resumen Técnico)

El sistema adopta una **Arquitectura Multicapa Desacoplada**:





Resumen del Esquema de Base de Datos

La persistencia de datos cuenta con las siguientes entidades principales (ver detalle completo de tablas e índices en SPEC.md):

- * **ingredients** y **categories**: Registro normalizado de ingredientes con perfiles organolépticos en formato JSONB.
- * **flavor_pairings**: Grafo de aristas que almacena la afinidad (0.00 a 1.00), razonamiento pre-calculado, tipo de fuente de datos y puntaje de confianza.
- * **users** y **revoked_tokens**: Gestión de cuentas de usuario y lista de revocación de tokens JWT.
- * **recipes** y **recipe_search_cache**: Repositorio permanente de recetas con caché indexado por hash SHA-256 e inyección de sanitización de payload (máximo 30 KB por receta).
- * **pairing_review_queue**: Cola de revisión para el control de alucinaciones y arbitraje manual.

Endpoints REST Principales

- GET /api/v1/graph: Subgrafo paginado para el lienzo del grafo (limit, offset, min_affinity).
- POST /api/v1/pairings/evaluate: Cálculo determinístico de la matriz de sinergia N x N, score global y elementos discordantes.
- POST /api/v1/ai/explain-pairing: Generación en tiempo real de explicaciones organolépticas.
- POST /api/v1/ai/suggest-replacement: Sugerencia generativa de ingredientes alternativos para corregir choques de sabor.

4. Experiencia de Usuario y Modelo Freemium

4.1 Lógica del Grafo Progresivo

- **1 Ingrediente:** Nodo central con aristas radiales a sus Top 5-8 vecinos de mayor afinidad.
- **2 Ingredientes:** Arista de afinidad coloreada (Verde para afinidad >75%) e iluminación intensa (100% opacidad) en los ingredientes vecinos que combinan bien con ambos.
- **Ingrediente Discordante:** Arista en **ROJO punteado** (<45%) y atenuación visual (30% opacidad) en los nodos periféricos.

4.2 Modelo de Suscripción (Tiers de Servicio)

- **Tier Gratuito (Free):** Evaluación de hasta **3 ingredientes por búsqueda** (ideal para triadas culinarias como la *Caprese* o *Mirepoix*), visualización del grafo y guardado de combinaciones favoritas.
- **Tier Pago (Pro):** Evaluación de hasta **10 ingredientes**, explicaciones ilimitadas con IA, guardado de recetas completas con notas personales y exportación a JSON y API Key.

5. Plan de Implementación Ágil (Resumen de 3 Meses)

El desarrollo del proyecto se estructura en **3 Meses (12 Semanas)** divididos en **6 Sprints ágiles de 2 Semanas** (ver mapa detallado y *Definition of Done* en PLAN.md):

MES 1: Cimientos, Pipeline de Datos & Prototipado
- Sprint 1: DDL Postgres, Redis, Alembic & Entorno Docker
- Sprint 2: Seed LLM (--dry-run) & Spike Grafo React 2D
MES 2: Backend Core, Auth & Caché de Recetas
- Sprint 3: REST API Grafo, Sinergia N x N & Pytest TDD
- Sprint 4: Auth Argon2id, Redis Blacklist & Spoonacular 30k
MES 3: Frontend Progresivo, IA Online & Defensa Académica
- Sprint 5: UI Grafo 2D, Intensidad Armónica & Drawers
- Sprint 6: IA Online Fallbacks, CI/CD, QA & Tesis

6. Viabilidad Económica

Nivel de Escala	Usuarios Activos / Mes	API Recetas (Con Caché DB)	API IA (Gemini Flash / GPT-4o-mini)	Hosting & Post-greSQL	Costo Total Estimado
MVP / Defensa Tesis	1 - 100	\$0.00 (Free Tier)	\$0.00 (Free Tier)	\$0.00 (Render / Supabase Free)	\$0.00 USD / mes
Producción Inicial	1,000	\$0.00 (Caché DB absorbe 95%)	~\$0.15 USD	\$0 - \$5.00 USD	~ \$0.15 - \$5.00 USD / mes
Escala Media	25,000	~\$29.00 USD (Spoonacular Builder)	~\$2.50 USD	~\$10.00 USD (DB 5GB)	~ \$41.50 USD / mes

Referencias a la Documentación Técnica Completa

Para profundizar en los aspectos específicos de implementación y código:

- * **SPEC.md:** Especificación técnica detallada con el DDL completo, scripts de sanitización (`sanitize_recipe_payload`), suite anti-alucinaciones (`verify_coherence.py`), Redis token revocation, contratos Pydantic v2 y pipeline de CI/CD en GitHub Actions.
- * **PLAN.md:** Plan de implementación detallado sprint por sprint, secuencia de kickoff en 5 pasos y archivos de infraestructura (`.env.example`, `docker-compose.yml`, `scripts/backup.sh`).
- * **README.md:** Portada del repositorio y guía de arranque rápido para desarrollo local.